Django and PostgreSQL Web Application Proposal

Hello,

My name is Fabricio Santana. I am a software engineer and technical lead with 20+ years of experience in backend development, software architecture, database integration, automated testing, cloud computing, and delivery of critical systems. I have also led large engineering teams and worked on projects where data consistency, business rules, validation, security, and maintainability are essential.

Your project is a strong match for my background because the main challenge is not only building Django models and CRUD endpoints. The important part is defining a secure and maintainable application architecture where authentication, authorization, PostgreSQL data design, API contracts, responsive UI flows, validation, testing, and deployment readiness work together safely in production.

My relevant approach for this kind of project is to combine backend architecture, authentication design, database modeling, REST API design, responsive product delivery, deployment support, and automated testing into one coherent delivery plan. This keeps the first version practical and bounded while preserving a solid technical base for future expansion.

Working methodology

My delivery method is designed to reduce risk before development starts. The work begins with a fixed Discovery Sprint and then moves into a delivery package only after scope, priorities, responsibilities, dependencies, and acceptance criteria are clear.

The first step is a one-week Discovery Sprint with a fixed price of USD 500. In this phase, I can hold up to five remote meetings to understand the business problem, review the target workflow, analyze the required data model and user roles, map requirements, identify risks, review integration needs, and define priorities. The deliverable is a practical work plan for the implementation, including recommended scope, technical approach, milestones, success criteria, and the most appropriate delivery package.

After discovery, the project can move into one of three fixed-price, four-week delivery packages. The package is selected according to scope, complexity, delivery speed, integration needs, quality requirements, and the amount of engineering support required. Some roles may participate part-time depending on the agreed scope, especially Scrum Master, technical leadership, test automation, DevOps, observability, and deployment support.

Delivery package options

- **Essential Delivery** — USD 2,000, 4 weeks. Includes a Scrum Master and one full-stack engineer. This package is designed for a focused module, MVP, backend feature, import flow, or small integration with limited uncertainty. It is suitable when the scope is well defined, the number of integrations is limited, and the main goal is to deliver a reliable working feature without unnecessary overhead.
- **Product Acceleration** — USD 4,000, 4 weeks. Includes a Scrum Master, two full-stack engineers, and one engineer focused on test automation, DevOps, observability, and deployment. This package is recommended when faster delivery is needed or when the scope includes a product, API, integration, internal workflow, multiple features, or more demanding quality needs.

- **Scale & Intelligence** — USD 6,000, 4 weeks. Includes a Technical lead, Scrum Master, two full-stack engineers, and one engineer focused on test automation, DevOps, observability, and deployment. This package is designed for complex systems, advanced data workflows, automation, auditability, security, observability, broader platform evolution, or stronger technical scale planning.

Role responsibilities are clear: the Scrum Master organizes scope, communication, ceremonies, progress tracking, and delivery alignment; full-stack engineers implement backend, frontend, API, and database changes as needed; the test automation, DevOps, observability, and deployment engineer supports automated tests, CI/CD, monitoring, release quality, and deployment; and the Technical lead, included in the Scale & Intelligence package, owns architecture, technical decisions, code quality, and risk management for more complex projects.

For a Django application like this, the implementation setup can include authentication, authorization, domain modeling, PostgreSQL integration, API delivery, responsive frontend flows, testing, deployment support, and handoff work within the agreed package scope.

Each delivery cycle is executed with Scrum-inspired practices, frequent alignment, technical review, automated testing when applicable, and a demonstration of what was delivered. The client keeps ownership of the produced code, and the scope is agreed before execution to avoid open-ended work and unclear expectations.

Construction defect warranty. After delivery and acceptance, I include a 30-day warranty for construction defects related to the approved scope. This covers fixes for implementation errors, broken agreed behavior, or defects introduced by the delivered code. It does not cover new features, changes in business rules, new entities or workflows not included in the approved scope, third-party service changes, infrastructure changes outside the project, or requests that expand the original acceptance criteria.

Opportunity analysis

This project should be treated as an application architecture problem, not as a generic stack exercise. A robust Django application needs more than authentication and CRUD screens. It needs a clear domain model, safe authentication and authorization behavior, reliable API boundaries, responsive user flows, validation rules, migration discipline, testing strategy, and an operational path to deployment.

The business value is also important: the first version should deliver a production-capable application foundation that your team can extend safely without expensive rewrites. That means making explicit decisions about entities, roles, permissions, user journeys, API consumers, and release boundaries before implementation grows in the wrong direction.

The main technical risks are underdefining the domain model, underestimating permissions and authentication complexity, exposing APIs before the actual workflow is clear, creating weak validation around CRUD flows, leaving API pagination, filtering, permissions, and error formats undefined, and keeping deployment assumptions vague. Scope can also expand quickly into dashboards, reports, file uploads, search, notifications, or heavier frontend requirements if the first release boundary is not controlled.

A practical first implementation cycle should start with a bounded production-ready workflow: secure authentication, explicit authorization, the agreed PostgreSQL-backed entities, REST endpoints for the agreed scope, responsive interfaces for the main user journeys, automated tests for core behavior, and the technical foundation required for future expansion. The exact delivery boundary should be confirmed during discovery.

Proposed work plan

I would approach the project in structured phases. The Discovery Sprint is the immediate recommended commitment, and implementation can move directly into a fixed delivery cycle once the domain, workflow, API boundary, and acceptance criteria are confirmed.

1. Discovery, workflow definition, and technical design

- Review the business objective, target users, first release boundary, key entities, CRUD scope, and required user journeys.
- Define the authentication and authorization model, including login behavior, password recovery, roles, and permissions where needed.
- Define the PostgreSQL data model, API boundary, responsive UI approach, deployment model, security assumptions, and acceptance criteria for the first release.
- Produce a concrete implementation plan with architecture recommendation, responsibilities, dependencies, risks, test expectations, and package recommendation.

2. Core application architecture and backend foundation

- Set up the Django project structure, domain modules, environment configuration, database integration, and migration workflow.
- Implement the authentication foundation, permission model, password-reset behavior where required, and security-sensitive validation behavior.
- Define reusable service boundaries, serializers, API contracts, pagination/filtering rules where needed, and validation rules aligned with the agreed scope.
- Establish the technical base for deployment, secure configuration, and maintainable future evolution.

3. Domain model, CRUD workflows, and API delivery

- Implement the agreed domain entities, relationships, CRUD operations, and business-rule validation.
- Deliver REST endpoints for the agreed first workflow scope, including error handling, authorization controls, and consistent response patterns.
- Keep the first implementation cycle focused on the bounded release rather than uncontrolled feature expansion.
- Structure the application so future entities and workflows can be added without large architectural rewrites.

4. Responsive user interface and operational flows

- Implement the agreed responsive user flows for desktop and mobile usage.
- Ensure the authentication, CRUD interactions, and validation feedback work cleanly across target screen sizes.
- Refine the user-facing flows according to the agreed operational needs rather than decorative frontend complexity.
- Preserve a maintainable frontend structure consistent with the chosen Django delivery approach.

5. Testing, deployment support, documentation, and handoff

- Test authentication, permissions, CRUD operations, API behavior, validation rules, responsive flows, and edge cases.
- Add deployment support, migration guidance, and operational visibility where appropriate.
- Document the architecture, setup steps, release assumptions, and maintenance guidelines.
- Provide a handoff session or written notes so your team can run, extend, and maintain the application safely.

Estimated timeline

For the immediate next step, my recommendation is a 1-week Discovery Sprint. After discovery, the first implementation cycle can move into a fixed 4-week delivery package. For a bounded production-ready Django application with authentication, PostgreSQL, REST APIs, responsive UI, and testing, the likely recommendation is one Product Acceleration cycle. If the project needs broader role complexity, richer frontend behavior, stronger observability, heavier integration requirements, or a larger API surface, the likely recommendation is one Scale & Intelligence cycle.

A practical schedule would be:

- Week 1: Discovery Sprint, workflow mapping, domain modeling, authentication and permission model, API boundary, acceptance criteria, and implementation plan.
- Weeks 2 to 5: first fixed implementation cycle, covering the agreed first operational release and technical handoff.

If discovery shows that the first version is narrower than currently implied, the implementation may fit a smaller delivery shape. If it shows broader workflows, stronger security needs, more complex frontend demands, or additional integrations, I will recommend the larger package or additional cycles accordingly.

Availability and communication

I can work remotely and keep communication in English or Portuguese through Workana messages, scheduled calls, and written progress updates. I usually prefer short alignment checkpoints during the week, especially after reviewing the workflow and after each major implementation milestone.

This helps reduce misunderstandings and keeps the project aligned with your actual business rules, user journeys, release boundary, and operational constraints.

Budget

Based on the current understanding, my recommended immediate starting budget is USD 500 for the one-week Discovery Sprint. This gives you a concrete plan before committing to the full implementation budget, while keeping a clear path to delivery in the following implementation cycle.

After discovery, the first implementation cycle will likely fit one of these packages:

- **Product Acceleration** at USD 4,000 for 4 weeks if the scope is a bounded production-ready Django application with authentication, PostgreSQL, REST APIs, responsive UI, testing, and deployment support.
- **Scale & Intelligence** at USD 6,000 for 4 weeks if the scope includes broader permissions, stronger auditability, richer frontend complexity, more demanding security requirements, heavier integrations, or larger workflow scope.

This assumes that the client provides timely access to business requirements, examples of the main workflows, approval on the first release boundary, and the target hosting or deployment environment. It also assumes the first release focuses on the agreed core entities and user journeys rather than broader expansion into reporting, advanced search, notifications, file uploads, third-party integrations, or unrelated modules unless those items are confirmed during discovery.

The Discovery Sprint will refine the scope and confirm the correct implementation package. If the review shows a narrower and tightly bounded workflow, I can recommend a smaller delivery shape. If it shows broader application scope, higher security requirements, or additional integrations, I will recommend the larger package or additional cycles.

Questions before final confirmation

Before confirming the final scope, I would like to clarify a few points:

1. What business problem should the first version solve, and what are the main user journeys?
2. What core entities and CRUD operations are in scope for the first release?
3. What authentication model is required: basic login, email verification, password reset, role-based access, social login, or another model?
4. Are there different user roles and permissions that need to be supported from day one?
5. Will the REST API be used only by this application, or also by third-party clients or mobile apps?
6. Should the frontend be implemented with Django templates and server-rendered flows, or is a richer frontend layer expected?
7. What responsive views or screens matter most for mobile usage?
8. What hosting or deployment environment is expected, and who will own infrastructure after launch?
9. What level of automated testing and documentation is expected for acceptance?
10. Is there already an approved budget range and target date for the first operational release?

Once these points are clear, I can confirm the final delivery plan and milestones.